МЕТОДИ РОЗРОБКИ ПАРАЛЕЛЬНИХ АЛГОРИТМІВ

Проблема створення паралельного ПЗ

- Створення та розробка паралельних алгоритмів є одночасно *науковою та інженерною проблемою*, бо вимагає від розробника точності математика, майстерності інженера і досвіду вченого з суміжних дисциплін.
- Організація паралельних обчислень визначається, перш за все, методом *декомпозиції* розв'язуваної задачі на підзадачі, що в свою чергу визначає вибір структурних даних, засобів паралельного програмування, методів взаємодії і т. і.
- Нижче розглядаються декілька основних способів декомпозиції, що є найбільш уживаними при проектуванні паралельних алгоритмів.

Метод балансового дерева

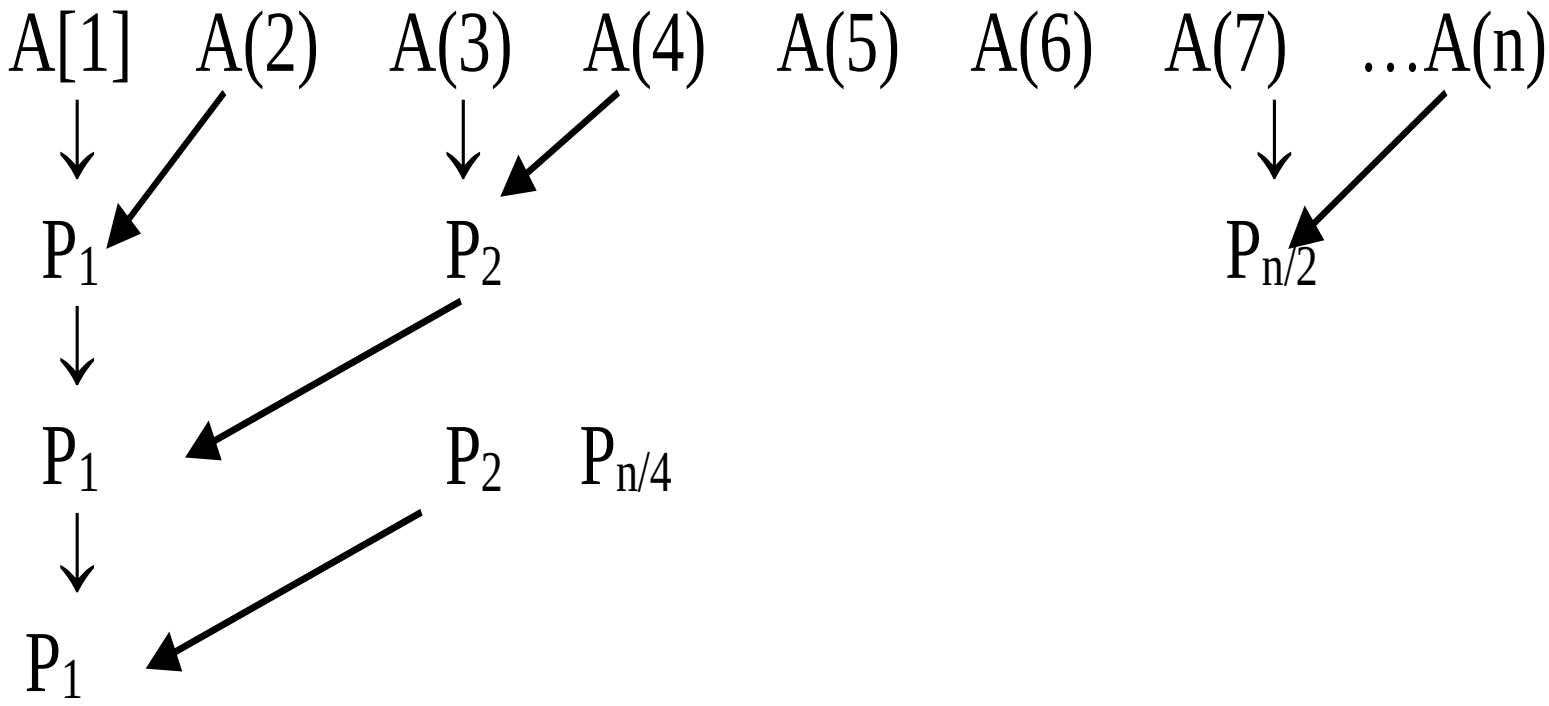
- При цій стратегії *декомпозиція задачі подається у вигляді дерева*, в якому вершина (корінь) відповідає самій задачі, проміжні вершини – підзадачам, а термінальні вершини – вхідним даним.
- Дерево підзадач – *балансоване*, якщо кількість вершин у піддеревах, одного рівня ієрархії приблизно однакова; тоді можна очікувати приблизно однаковий час виконання паралельних гілок алгоритму і, значить, мінімальні непродуктивні витрати часу.
- Саме ця властивість дозволяє ефективно застосовувати розпаралелювання обчислень.
- Основними структурами даними для таких задач є вектори, масиви або списки.

Приклад – сумування елементів вектора (1)

- Для прикладу розглянемо задачу обчислення суми $n=2^k$ чисел, що є елементами масиву:

$$S = \sum_{i=1}^{n} A[i]$$

- Нижче на рисунку показана “логарифмічна” схема процесів попарного сумування і передач даних між ними:

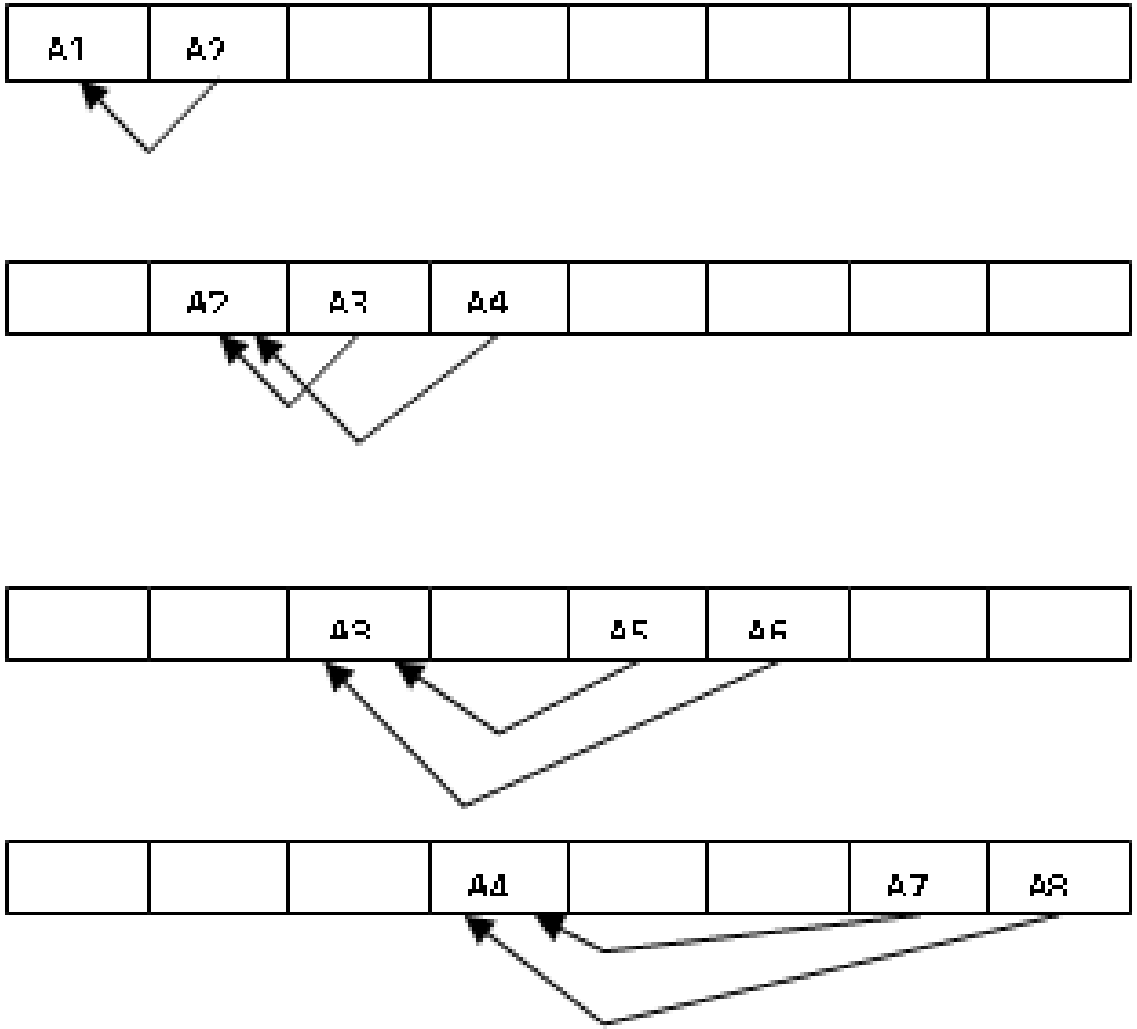


Приклад – сумування елементів вектора (2)

- Паралельний алгоритм для вирішення цієї задачі має такий вигляд:
  - <u>for</u> i=1 <u>to</u> k <u>do</u>
  - <u>for</u> $1 \leq j \leq n/2^i$ <u>doall</u>
  - A[j]:=: A[2j-1] + A[2j];
  - <u>endfor</u>;
  - <u>endfor</u>;
  - S:= A[1];

Приклад – сумування елементів вектора (3)

- На малюнку подано випадок $n=2^3=8$. P_i читає A[2i-1] та A[2i] і посилає суму цієї пари чисел до A[i]. Картина передач після першого етапу підсумувань показана нижче.



Засоби паралельного програмування (1)

- Текст цієї програми містить *два засоби паралельного програмування*, які відрізняються від звичних операторів послідовних програм.
- Перший з них – оператор паралельного циклу виду *for <умова> doall*. що задає паралельне виконання всіх ітерацій, визначених в *<умові>,* одночасно.
- Це означає, зокрема, що ітерації цього циклічного оператора повинні бути *інформаційно незалежними.*
- умовою інформаційної незалежності двох операторів P і Q, для яких визначені множини вхідних і вихідних змінних, відповідно, in(P), in(Q), out(P) і out(Q), є умова пустоти множини (*умова Рассела-Бернштейна*):

$$in(P) \cap out(Q) \cup in(Q) \cap out(P) \cup out(P) \cap out(Q)=\varnothing$$

Засоби паралельного програмування (2)

- Інший паралельний оператор – *мультиприсвоювання*, позначений у програмі ":=:"
- Семантика цього оператора визначає *паралельне виконання певної множини присвоювань* (в даному випадку, визначеною в <*умові*>) *у два етапи.*
- На першому етапі обчислюються значення правих частин <u>всіх</u> присвоювань, а на другому <u>одночасно</u> відбувається власне присвоювання значень, обчислених на першому етапі, відповідним змінним, визначеним лівими частинами операторів присвоювання.
- Саме *двоетапність мультиприсвоювання*, а також синхронізованість обчислень у моделі PRAM *гарантують інформаційну незалежність ітерацій* паралельного циклічного оператора.

Аналіз продуктивності алгоритму (1)

- Часова складність алгоритму визначається висотою дерева процесів і складає величину $T_{n/2} = \log_2 n + 1$, що має *логарифмічний порядок* $O(\log_2 n)$.
- Програма максимально використовує n/2 процесорів, тому *прискорення* складає $S_{n/2} = n+1/(\log_2 +1)$.
- При цьому ефективність паралельних обчислень складає $E_{n/2} = 2(n+1)/n(\log_2 n+1)$.
- Звідси ціна паралельних обчислень визначається як $C_{n/2} = \sum_{i=1}^{\log_2 n} \left(\frac{n}{2^i} \right) + 1$
- що за порядком O(n) збігається з величиною часової складності послідовного алгоритму. Отже, цей алгоритм є оптимальним.

Аналіз продуктивності алгоритму (2)

- До такого типу паралельних алгоритмів належать і інші алгоритми, що вимагають бінарних операцій над числами — елементами масиву, наприклад, знаходження найменшого чи найбільшого елемента масиву.
- Практично, коли $p<n/2$, за принципом Брента маємо $T_p=O(c/p+T_\infty)=O(n/p+\log_2 n)$.
- Хоча розглянута вище задача обчислення суми елементів числового масиву виглядає як така, що відповідає парадигмі паралелізму за даними, метод балансованого дерева тісно не пов’язаний з цією схемою обчислень і може бути реалізований у контексті *будь якої парадигми паралельного програмування*.

Метод "розподіляй і владарюй"

- В загальному вигляді стратегія "розподіляй і владарюй" полягає у:
  1. розподілі первинної задачі на декілька підзадач приблизно рівної складності;
  2. рекурсивно-парлельного вирішення підзадач, виділених на попередньому кроці;
  3. використанні результатів підзадач, як аргументів, для розв′язку первинної задачі.
- Простим прикладом для такого підходу може бути задача множення двох числових матриць A[q*r] і B[r*s]. Елементи матриці добутку визначаються за формулою:
- де обчислення елементів результуючої матриці $C_{ij} = \sum_{k=1}^{r} A_{ik} * B_{kj}$
- не залежать одне від одного. Тому в даному випадку
- первинна задача просто розпадається на q*s паралельних підзадач.

Паралельна програма множення матриць

- Маючи qs процесорів, паралельний алгоритм матиме вигляд:

```
• C_ij = 0;
• for i = 1 to q doall
•     for j = 1 to  s doall
•         for k = 1 to r do
•             C[i,j] := C[i,j] + A_ik * B_kj;
•         endfor;
•     endfor;
• endfor;
```

Аналіз продуктивності алгоритму

- Часова складність складає величину: $T_1=qsr$, $T_{qs}=r$.
- Звідси показники продуктивності мають такі значення: $S_{qs}=qs$; $E_{qs}=qs/qs=1$, і отже з точки зору моделі PRAM алгоритм має *ідеальну ефективність*.
- Тому не дивно, що з точки зору його ціни алгоритм є *оптимальним*: $C_{qs}=T_{qs}*qs = qsr = T_1$.
- Якщо до внутрішнього, послідовного, циклу даної програми застосувати метод балансованого дерева, то можна одержати *більше прискорення*:

$$S_{qrs} = qsr/\log_2 r$$

- Метод “розподіляй і владарюй” в загальному випадку найбільше відповідає схемі обчислень *паралелізму за керуванням*. Проте в простіших випадках цей метод може бути реалізований у рамках і інших парадигм паралельного програмування.